

Python Strings

A complete reference for Python 3.14 — from fundamentals to production patterns

What is a String?

Any sequence of characters enclosed within single quotes or double quotes is called a String.

```
s1 = 'durga'
s2 = "durga"
```

Note: In most other languages like C, C++, and Java, a single character within single quotes is treated as a `char` data type. However, in Python there is no `char` type — even a single character is treated as a `str`.

```
ch = 'a'
print(type(ch))  # <class 'str'>
```

Multi-line String Literals

We can define multi-line string literals using triple single quotes or triple double quotes.

```
s = '''Rohit is one of the
best cricket players in the world.'''
```

We can also use triple quotes to include single quotes or double quotes as symbols inside the string literal.

LITERAL	VALID?	REASON
'This is ' single quote'	No	Unescaped inner single quote
'This is \' single quote'	Yes	Escaped with backslash
"This is ' single quote"	Yes	Double quotes wrap single
'This is " double quote'	Yes	Single quotes wrap double
'The "Python Notes" by 'durga''	No	Inner single quotes clash
'The \"Python Notes\" by \'durga\''	Yes	Both escaped
'''The "Python Notes" by 'durga' is helpful'''	Yes	Triple quotes wrap both

Accessing Characters of a String

We can access characters of a string in two ways:

- By using index
- By using the slice operator

Accessing Characters by Index

Python supports both positive (+ve) and negative (-ve) indices.

- Positive index goes from left to right (forward direction).
- Negative index goes from right to left (backward direction).

```
s = 'durga'
print(s[0])    # 'd'
print(s[4])    # 'a'
print(s[-1])   # 'a'
print(s[-5])   # 'd'
```

Note: If we try to access a character with an out-of-range index, we will get an `IndexError`.

```
s = 'durga'
# print(s[10]) # IndexError: string index out of range
```

Program — display each character with its positive and negative index:

```
s = input("Enter some string: ")
i = 0
for x in s:
    print("positive index {} and negative index {} is: {}".format(i, i - len(s),
x))
    i += 1
```

```
Enter some string: durga
positive index 0 and negative index -5 is: d
positive index 1 and negative index -4 is: u
positive index 2 and negative index -3 is: r
positive index 3 and negative index -2 is: g
positive index 4 and negative index -1 is: a
```

Accessing Characters by the Slice Operator

Syntax:

```
s[begin : end : step]
```

- `begin` — starting index of the slice (inclusive).
- `end` — the slice stops at `end - 1` (exclusive).
- `step` — the increment value. Default is `1`.

Note:

- If `begin` is not given, slicing starts from the beginning of the string.
- If `end` is not given, slicing continues to the end of the string.
- If `step` is `0`, Python raises a `ValueError`.

```
s = "Learning Python is very very easy!!!"

print(s[1:7:1])    # 'earnin'
print(s[1:7])      # 'earnin'
print(s[1:7:2])    # 'eri'
print(s[:7])       # 'Learnin'
print(s[7:])       # 'g Python is very very easy!!!'
print(s[::])       # 'Learning Python is very very easy!!!'
print(s[:])        # 'Learning Python is very very easy!!!'
print(s[::-1])     # '!!!ysae yrev yrev si nohtyP gninraeL'
```

Behaviour of the slice operator:

- If `step` is positive, slicing goes forward (left to right) from `begin` to `end - 1`.
- If `step` is negative, slicing goes backward (right to left) from `begin` to `end + 1`.

Default values:

DIRECTION	BEGIN	END	STEP
Forward	0	length of string	+1
Backward	-1	-(length + 1)	-1

Slice case study:

```
s = 'deepleaningrocks'

print(s[1:6:2])      # 'epe'
print(s[::1])        # 'deepleaningrocks'
print(s[::-1])       # 'skcorgninaelpeed'
print(s[3:7:-1])     # '' (empty -- begin < end with negative step)
print(s[7:3:-1])     # 'ning'
print(s[0:10000:1])  # 'deepleaningrocks' (out-of-range end is clipped)
print(s[-4:1:-1])    # 'orgninaelpe'
print(s[-4:1:-2])    # 'ogiae'
print(s[10000:2:-1]) # 'skcorgninaelp'
# s[9:0:0]           # ValueError: slice step cannot be zero
```

Mathematical Operators for Strings

We can apply two mathematical operators on strings:

- `+` operator for concatenation.
- `*` operator for repetition.

```
print("durga" + "soft")  # durgasoft
print("durga" * 3)       # durgadurgadurga
```

Note:

- For `+`, both arguments must be of type `str`.
- For `*`, one argument must be `str` and the other must be `int`.

`len()` built-in function:

We can use `len()` to find the number of characters in a string.

```
s = 'durga'
print(len(s))  # 5
```

Program — traverse a string forward and backward using a while loop:

```
s = "Learning Python is very easy!!!"
n = len(s)
```

```
print("Forward direction:")
i = 0
while i < n:
    print(s[i], end=' ')
    i += 1

print("\nBackward direction:")
i = -1
while i >= -n:
    print(s[i], end=' ')
    i -= 1
```

Alternative ways using a for loop and slicing:

```
s = "Learning Python is very easy!!!"

# Forward using for loop
for ch in s:
    print(ch, end=' ')

# Forward using slice
for ch in s[:]:
    print(ch, end=' ')

# Backward using slice
for ch in s[::-1]:
    print(ch, end=' ')
```

Checking Membership

We can check whether a character or substring is present in a string using `in` and `not in` operators.

```
s = 'durga'
print('d' in s)      # True
print('z' in s)      # False
print('z' not in s)  # True
```

```
s = input("Enter main string: ")
subs = input("Enter sub string: ")

if subs in s:
    print(subs, "is found in the main string")
else:
    print(subs, "is not found in the main string")
```

Comparison of Strings

We can use comparison operators (`<`, `<=`, `>`, `>=`) and equality operators (`==`, `!=`) to compare strings. Comparison is done based on alphabetical (Unicode) order.

```
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

if s1 == s2:
    print("Both strings are equal")
elif s1 < s2:
    print("First string is less than second string")
else:
    print("First string is greater than second string")
```

```
# Simple comparisons
print("apple" == "apple")  # True
print("apple" < "banana")  # True (a comes before b)
print("Zebra" < "apple")   # True (uppercase letters have lower Unicode values)
```

Removing Spaces from a String

We can use the following methods to remove whitespace:

- `strip()` — removes spaces from both left and right sides.
- `lstrip()` — removes spaces from the left side only.
- `rstrip()` — removes spaces from the right side only.

```
s = "    hello world    "
print(s.strip())    # 'hello world'
print(s.lstrip())   # 'hello world    '
print(s.rstrip())   # '    hello world'
```

```
# Practical use: sanitise user input before comparing
username = input("Enter username: ")
cleaned = username.strip()

if cleaned == "admin":
    print("Welcome, admin!")
else:
    print("Access denied for user:", cleaned)
```

```
# strip() also removes custom characters from both ends
filename = "###report.pdf###"
print(filename.strip("#")) # 'report.pdf'
```

Finding Substrings

We have four methods for finding substrings:

METHOD	DIRECTION	IF NOT FOUND
<code>find()</code>	Forward (left to right)	Returns -1
<code>index()</code>	Forward (left to right)	Raises <code>ValueError</code>
<code>rfind()</code>	Backward (right to left)	Returns -1
<code>rindex()</code>	Backward (right to left)	Raises <code>ValueError</code>

`find()` Method

`s.find(substring)` returns the index of the first occurrence of the substring. Returns -1 if not found.

```
s = "Learning Python is very easy"

print(s.find("Python")) # 9
```

```
print(s.find("Java"))      # -1
print(s.find("r"))        # 3
print(s.rfind("r"))       # 21
```

We can also specify a search range: `s.find(substring, begin, end)` — searches from `begin` to `end - 1`.

```
s = "durgaravipavanshiva"

print(s.find('a'))          # 4
print(s.find('a', 7, 15))  # 10
print(s.find('z', 7, 15))  # -1
```

index() Method

The `index()` method works exactly like `find()` except it raises a `ValueError` when the substring is not found instead of returning -1.

```
s = input("Enter main string: ")
subs = input("Enter sub string: ")

try:
    n = s.index(subs)
except ValueError:
    print("Substring not found")
else:
    print("Substring found at index", n)
```

Program — Find All Positions of a Substring

```
s = input("Enter main string: ")
subs = input("Enter sub string: ")

flag = False
pos = -1
n = len(s)

while True:
    pos = s.find(subs, pos + 1, n)
    if pos == -1:
        break
    print("Found at position", pos)
```



```
flag = True

if not flag:
    print("Not found")
```

```
Enter main string: Mississippi
Enter sub string: issi
Found at position 1
Found at position 4
```

Counting Substrings

`s.count(substring)` returns the number of non-overlapping occurrences of the substring in `s`.

`s.count(substring, begin, end)` limits the search from `begin` to `end - 1`.

```
s = "abcabcabcabcadda"

print(s.count('a'))      # 6
print(s.count('ab'))     # 4
print(s.count('a', 3, 7)) # 2
```

```
# Count how many times a word appears in a sentence
sentence = "to be or not to be that is the question to ponder"
print(sentence.count("to")) # 3
```

Replacing Substrings

`s.replace(old, new)` returns a new string with every occurrence of `old` replaced by `new`. The original string is not changed.

```
s = "Learning Python is very difficult"
s1 = s.replace("difficult", "easy")
print(s1) # Learning Python is very easy
```

```
s = "ababababababab"
s1 = s.replace("a", "b")
print(s1)  # bbbbbbbbbbbbbbb
```

String immutability — `replace()` creates a new object:

```
s = "abab"
s1 = s.replace("a", "b")

print(s, "is at id:", id(s))  # 'abab' -- original unchanged
print(s1, "is at id:", id(s1))  # 'bbbb' -- new object
```

Note: `id()` shows that `s` and `s1` are different objects in memory. The original string is always preserved.

Splitting Strings

`s.split(separator)` splits a string into a list. The default separator is any whitespace.

```
s = "durga software solutions"
l = s.split()
for x in l:
    print(x)
# durga
# software
# solutions
```

```
s = "22-02-2018"
l = s.split('-')
for x in l:
    print(x)
# 22
# 02
# 2018
```

Joining Strings

`separator.join(group)` joins a list or tuple of strings into one string using the separator between each element.

```
# Joining a tuple
t = ('karan', 'arjun', 'rohit')
s = '-'.join(t)
print(s)  # karan-arjun-rohit
```

```
# Joining a list
l = ['hyderabad', 'mumbai', 'bangalore', 'pune']
s = ':'.join(l)
print(s)  # hyderabad:mumbai:bangalore:pune
```

Changing Case

METHOD	EFFECT
<code>upper()</code>	Converts all characters to uppercase
<code>lower()</code>	Converts all characters to lowercase
<code>swapcase()</code>	Converts lowercase to uppercase and vice versa
<code>title()</code>	Converts the first character of each word to uppercase
<code>capitalize()</code>	Converts only the first character of the string to uppercase

```
s = "learning Python is very Easy"

print(s.upper())    # LEARNING PYTHON IS VERY EASY
print(s.lower())    # learning python is very easy
print(s.swapcase()) # LEARNING pYTHON IS VERY eASY
print(s.title())    # Learning Python Is Very Easy
print(s.capitalize()) # Learning python is very easy
```

Checking Start and End of a String

`s.startswith(substring)` and `s.endswith(substring)` return `True` or `False`.

```
s = "learning Python is very easy"
```

```
print(s.startswith("learning"))  # True
print(s.startswith("Python"))    # False
print(s.endswith("easy"))        # True
print(s.endswith("learning"))    # False
```

```
# Both methods also accept a tuple of values
```

```
filename = "report_2025.pdf"
```

```
print(filename.endswith((".pdf", ".docx", ".txt")))  # True
print(filename.startswith(("report", "summary")))    # True
```

Checking Type of Characters

METHOD	RETURNS TRUE WHEN
<code>isalnum()</code>	All characters are letters or digits (a-z, A-Z, 0-9)
<code>isalpha()</code>	All characters are alphabets
<code>isdigit()</code>	All characters are digits
<code>islower()</code>	All cased characters are lowercase
<code>isupper()</code>	All cased characters are uppercase
<code>istitle()</code>	String is in title case
<code>isspace()</code>	String contains only whitespace

```
print('Durga786'.isalnum())      # True
print('durga'.isalpha())         # True
print('durga786'.isalpha())      # False
print('786786'.isdigit())        # True
print('abc'.islower())           # True
print('ABC'.isupper())           # True
print('Learning Python Is Easy'.istitle())  # True
print('Learning Python is Easy'.istitle())  # False
print('      '.isspace())        # True
```

Demo program — classify a character entered by the user:

```
s = input("Enter any character: ")

if s.isalnum():
    print("Alpha Numeric character")
    if s.isalpha():
        print("Alphabet character")
        if s.islower():
            print("Lowercase alphabet")
        else:
            print("Uppercase alphabet")
    else:
        print("It is a digit")
elif s.isspace():
    print("It is a space character")
else:
    print("Non-space special character")
```

```
Enter any character: a
Alpha Numeric character
Alphabet character
Lowercase alphabet
```

Formatting Strings

The main purpose of string formatting is to combine variables and string text into a meaningful output.

Python provides three main approaches:

APPROACH	SYNTAX	NOTES
f-string	<code>f"value is {x}"</code>	Fastest; evaluate at point of definition
<code>str.format()</code>	<code>"value is {}".format(x)</code>	Flexible; supports reusable templates
<code>%</code> -formatting	<code>"value is %s" % x</code>	Legacy style; avoid in new code

Case 1 — Basic formatting: default, positional, and keyword arguments

```
name    = 'durga'
salary = 10000
age     = 48

print("{}'s salary is {} and his age is {}".format(name, salary, age))
print("{0}'s salary is {1} and his age is {2}".format(name, salary, age))
print("{x}'s salary is {y} and his age is {z}".format(z=age, y=salary, x=name))
```

```
durga's salary is 10000 and his age is 48
durga's salary is 10000 and his age is 48
durga's salary is 10000 and his age is 48
```

Case 2 — Formatting numbers

Format codes for numbers:

CODE	MEANING
<code>d</code>	Decimal integer
<code>f</code>	Fixed-point float (default 6 decimal places)
<code>b</code>	Binary
<code>o</code>	Octal
<code>x</code>	Hexadecimal (lowercase)
<code>X</code>	Hexadecimal (uppercase)
<code>e</code>	Scientific notation

Decimal integer formatting:

```
print("The integer number is: {}".format(123))
print("The integer number is: {:d}".format(123))
print("The integer number is: {:5d}".format(123))    # min width 5
print("The integer number is: {:05d}".format(123))  # zero-padded width 5
```

```
The integer number is: 123
The integer number is: 123
The integer number is:  123
The integer number is: 00123
```

Float formatting:

```
print("The float number is: {}".format(123.4567))
print("The float number is: {:.f}".format(123.4567))
print("The float number is: {:.8.3f}".format(123.4567))
print("The float number is: {:08.3f}".format(123.4567))
print("The float number is: {:08.3f}".format(786786123.45))
```

```
The float number is: 123.4567
The float number is: 123.456700
The float number is:  123.457
The float number is: 0123.457
The float number is: 786786123.450
```

Note about `{:08.3f}`:

- Total minimum width is 8.
- Exactly 3 digits after the decimal point (rounded if needed).
- If total digits are fewer than 8, zeros are placed at the start.
- If the number is larger than 8 positions, all digits are shown anyway.

Binary, octal, and hexadecimal:

```
print("Binary form:      {:b}".format(153))
print("Octal form:       {:o}".format(153))
print("Hex form (lower): {:x}".format(154))
print("Hex form (upper): {:X}".format(154))
```

```
Binary form:      10011001
Octal form:       231
Hex form (lower): 9a
Hex form (upper): 9A
```

Note: Binary, octal, and hexadecimal formatting only works with integer values, not floats.

Case 3 — Signed numbers

```
print("int value with sign:  {:+d}".format(123))
print("int value with sign:  {:+d}".format(-123))
print("float value with sign: {:.+f}".format(123.456))
print("float value with sign: {:.+f}".format(-123.456))
```

```
int value with sign:  +123
int value with sign:  -123
float value with sign: +123.456000
float value with sign: -123.456000
```

Case 4 — Alignment

SYMBOL	ALIGNMENT
<	Left
>	Right (default for numbers)
^	Centre
=	Sign placed at leftmost position

```
print("{:5d}".format(12))      # 12
print("{:<5d}".format(12))     # 12
print("{:>5d}".format(12))     # 12
print("{:^5d}".format(12))     # 12
print("{:=5d}".format(-12))    # - 12
print("{:^10.3f}".format(12.23456)) # 12.235
print("{:=8.3f}".format(-12.23456)) # - 12.235
```

Case 5 — String alignment with format()

For strings, the default alignment is left (opposite to numbers, which default to right).

```
print("{:5s}".format("rat"))   # 'rat '
print("{:>5s}".format("rat"))  # ' rat'
print("{:<5s}".format("rat"))  # 'rat '
print("{:^5s}".format("rat"))  # ' rat '
```



```
print("{:*^5}".format("rat"))  # '*rat*'
```

Case 6 — Truncating strings

```
print("{:.3}".format("durgasoftware"))  # 'dur'  
print("{:5.3}".format("durgasoftware"))  # 'dur  '  
print("{:>5.3}".format("durgasoftware"))  # '    dur'  
print("{:^5.3}".format("durgasoftware"))  # '  dur  '  
print("{:*^5.3}".format("durgasoftware"))  # '*dur*'
```

Case 7 — Formatting dictionary members

```
person = {'age': 48, 'name': 'durga'}  
print("{p[name]}s age is: {p[age]}".format(p=person))  
# durga's age is: 48  
  
# More convenient -- unpack with **  
print("{name}s age is: {age}".format(**person))  
# durga's age is: 48
```

Case 8 — Formatting class attributes

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
p1 = Person('durga', 48)  
p2 = Person('Harsha', 32)  
  
print("{p.name}s age is: {p.age}".format(p=p1))  
print("{p.name}s age is: {p.age}".format(p=p2))
```

```
durga's age is: 48  
Harsha's age is: 32
```

Case 9 — Dynamic formatting

```
string = "{:{fill}{align}{width}}"

print(string.format('cat', fill='*', align='^', width=5))    # *cat*
print(string.format('cat', fill='*', align='^', width=6))    # *cat**
print(string.format('cat', fill='*', align='<', width=6))    # cat***
print(string.format('cat', fill='*', align='>', width=6))    # ***cat
```

Case 10 — Dynamic float template

```
num = "{:{align}{width}.{precision}f}"

print(num.format(123.236, align='<', width=8, precision=2))  # '123.24  '
print(num.format(123.236, align='>', width=8, precision=2))  # '   123.24'
```

Case 11 — Formatting dates

```
import datetime

date = datetime.datetime.now()
print("It's now: {:%d/%m/%Y %H:%M:%S}".format(date))
# It's now: 02/01/2025 22:56:03
```

Case 12 — Formatting complex numbers

```
c = 1 + 2j
print("Real part: {0.real} Imaginary part: {0.imag}".format(c))
# Real part: 1.0 Imaginary part: 2.0
```

f-strings — the modern standard

f-strings evaluate expressions at the point of use. They are the fastest and most readable approach.

```
name    = "Tanvi"
score   = 92.5
grade   = "A"

print(f"{name} scored {score:.1f} and got grade {grade}")
# Tanvi scored 92.5 and got grade A
```

```
# f-string with = specifier for quick debugging (Python 3.8+)
batch_size      = 64
learning_rate   = 0.001

print(f"{batch_size=} {learning_rate=}")
# batch_size=64 learning_rate=0.001
```

```
# Numeric formatting inside f-strings follows the same mini-language
price = 1234567.89
print(f"Price -- {price:,.2f}")      # Price -- 1,234,567.89
print(f"Hex   -- {255:#010x}")      # Hex   -- 0x000000ff
print(f"Sci   -- {0.000314:.2e}")   # Sci   -- 3.14e-04
```

```
# Python 3.12+: reuse the same quote type inside the expression
students = [{"name": "Arjun", "marks": 88}, {"name": "Drishya", "marks": 95}]
for s in students:
    print(f"Student: {s["name"]} Marks: {s["marks"]}")
```

t-strings — Template String Literals

Template strings (t-strings) were introduced in Python 3.14 via PEP 750. Unlike f-strings, they do not immediately produce a plain `str` — they return a `Template` object. This allows you to inspect and sanitise interpolated values before rendering, which is important for building SQL queries, HTML, or any output where unsanitised user input would be a security risk.

```
from string import Template, Interpolation

# A t-string looks like an f-string with a 't' prefix
name = "Rohit"
score = 95

tmpl = t"Student {name} scored {score} marks."
print(type(tmpl)) # <class 'string.Template'>

# Render it safely
result = "".join(
    item if isinstance(item, str) else str(item.value)
```

```
    for item in tpl
)
print(result)  # Student Rohit scored 95 marks.
```

```
# The key advantage: inspect values before rendering
def safe_render(tmpl: Template) -> str:
    parts = []
    for item in tpl:
        if isinstance(item, str):
            parts.append(item)
        elif isinstance(item, Interpolation):
            # Sanitise here before including the value
            val = str(item.value).replace("<", "&lt;").replace(">", "&gt;")
            parts.append(val)
    return "".join(parts)

user_input = "<script>alert('xss')</script>"
html = safe_render(t"<p>Welcome, {user_input}</p>")
print(html)
# <p>Welcome, &lt;script&gt;alert('xss')&lt;/script&gt;</p>
```

Important String Programs

Q1 — Reverse a given string

```
# Method 1: slice (most idiomatic)
s = input("Enter some string: ")
print(s[::-1])
```

```
# Method 2: reversed() with join
s = input("Enter some string: ")
print(''.join(reversed(s)))
```

```
# Method 3: while loop
s = input("Enter some string: ")
i = len(s) - 1
result = ''
while i >= 0:
```

```
    result = result + s[i]
    i -= 1
print(result)
```

Q2 — Reverse the order of words

Input: Learning Python is very easy

Output: easy very is Python Learning

```
s = input("Enter some string: ")
l = s.split()
l1 = []
i = len(l) - 1
while i >= 0:
    l1.append(l[i])
    i -= 1
print(' '.join(l1))
```

Q3 — Reverse the content of each word

Input: Durga Software Solutions

Output: agruD erawtfoS snoituloS

```
s = input("Enter some string: ")
l = s.split()
l1 = []
i = 0
while i < len(l):
    l1.append(l[i][::-1])
    i += 1
print(' '.join(l1))
```

Q4 — Print characters at even and odd positions

```
# Method 1: using slicing
s = input("Enter some string: ")
print("Even position chars:", s[0::2])
print("Odd position chars:", s[1::2])
```

```
# Method 2: using a while loop
s = input("Enter some string: ")

print("Even position chars:")
i = 0
while i < len(s):
    print(s[i], end=',')
    i += 2

print("\nOdd position chars:")
i = 1
while i < len(s):
    print(s[i], end=',')
    i += 2
```

Q5 — Merge two strings character by character alternately

Input: s1 = "ravi", s2 = "reja"
Output: rreavjia

```
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

output = ''
i, j = 0, 0

while i < len(s1) or j < len(s2):
    if i < len(s1):
        output += s1[i]
        i += 1
    if j < len(s2):
        output += s2[j]
        j += 1

print(output)
```

Q6 — Sort: alphabets first, then digits

Input: B4A1D3

Output: ABD134

```
s = input("Enter some string: ")
s1 = s2 = output = ''

for x in s:
    if x.isalpha():
        s1 += x
    else:
        s2 += x

for x in sorted(s1):
    output += x
for x in sorted(s2):
    output += x

print(output)
```

Q7 — Expand run-length encoded string

Input: a4b3c2
Output: aaaabbbbcc

```
s = input("Enter some string: ")
output = ''
previous = ''

for x in s:
    if x.isalpha():
        output += x
        previous = x
    else:
        output += previous * (int(x) - 1)

print(output)
```

Q8 — Shift each letter by the following digit

Input: a4k3b2

Output: aeknbd

```
s = input("Enter some string: ")
output = ''
previous = ''

for x in s:
    if x.isalpha():
        output += x
        previous = x
    else:
        output += chr(ord(previous) + int(x))

print(output)
```

Q9 — Remove duplicate characters

Input: ABCDABBCDABBBCCCDDEEEF
Output: ABCDEF

```
s = input("Enter some string: ")
l = []

for x in s:
    if x not in l:
        l.append(x)

print(''.join(l))
```

Q10 — Count occurrences of each character

Input: ABCABCABBCDE
Output: A — 3 times, B — 4 times, C — 3 times, D — 1 time, E — 1 time

```
s = input("Enter some string: ")
d = {}

for x in s:
    if x in d:
```



```

        d[x] += 1
    else:
        d[x] = 1

for k, v in d.items():
    print("{} -- {} times".format(k, v))

```

Q11 — Reverse odd-indexed words only

Input: one two three four five six seven
 Output: one owt three ruof five xis seven

```

s = input("Enter some string: ")
l = s.split()
l1 = []
i = 0

while i < len(l):
    if i % 2 == 0:
        l1.append(l[i])
    else:
        l1.append(l[i][::-1])
    i += 1

output = ' '.join(l1)
print("Original:", s)
print("Output: ", output)

```

Palindrome Check

A palindrome is a string that reads the same forward and backward.

```

# Method 1: slice comparison
s = input("Enter a string: ")
if s == s[::-1]:
    print("Given string is a palindrome")
else:
    print("Given string is not a palindrome")

```

```
# Method 2: character-by-character loop
s = input("Enter a string: ")
is_palindrome = True

for i in range(len(s) // 2):
    if s[i] != s[-(i + 1)]:
        is_palindrome = False
        break

if is_palindrome:
    print("Given string is a palindrome")
else:
    print("Given string is not a palindrome")
```

```
# Method 3: recursive function
def is_palindrome(s):
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])

s = input("Enter a string: ")
if is_palindrome(s):
    print("Given string is a palindrome")
else:
    print("Given string is not a palindrome")

# Trace for "racecar":
# is_palindrome("racecar") → 'r'=='r' → is_palindrome("aceca")
# → 'a'=='a' → is_palindrome("cec")
# → 'c'=='c' → is_palindrome("e")
# → len("e") <= 1 → True
```

```
# Method 4: using reversed()
s = input("Enter a string: ")
if s == ''.join(reversed(s)):
```

```
print("Given string is a palindrome")
else:
    print("Given string is not a palindrome")
```

Common Mistakes

Mistake 1 — Forgetting that strings are immutable

```
# Wrong: replace() returns a new string; the original is unchanged
s = "status: pending"
s.replace("pending", "active")
print(s)  # 'status: pending' ← not changed

# Correct: reassign to capture the result
s = s.replace("pending", "active")
print(s)  # 'status: active'
```

Mistake 2 — Not checking the -1 return from find()

```
# Wrong: if substring is absent, find() returns -1.
# s[-1:] silently gives the last character -- not what you want.
text = "hello world"
idx = text.find("python")
result = text[idx:]  # gives 'd' -- wrong and silent

# Correct: always check for -1 before using the index
idx = text.find("python")
if idx != -1:
    result = text[idx:]
else:
    result = None
print(result)  # None
```

Mistake 3 — Concatenating strings in a loop with +

```
# Wrong: creates a new string object each iteration -- very slow for large data
words = ["one", "two", "three", "four", "five"]
result = ""
for w in words:
    result = result + " " + w
```

```
# Correct: collect into a list, then join -- much faster
result = " ".join(words)
print(result)  # 'one two three four five'
```

Mistake 4 — Comparing strings without normalising case

```
# Wrong: "Admin" != "admin" -- will miss valid values silently
role = "Admin"
if role == "admin":    # False
    print("access granted")

# Correct: normalise to a consistent case before comparing
if role.lower() == "admin":
    print("access granted")
```

Mistake 5 — index() without a try/except

```
# Wrong: raises unhandled ValueError when substring is not found
s = "hello world"
pos = s.index("python") # crashes the program

# Correct: handle the exception
try:
    pos = s.index("python")
except ValueError:
    print("substring not found")
```

Mistake 6 — Using step = 0 in a slice

```
s = "durga"
# s[:0] ← ValueError: slice step cannot be zero

# Correct: validate user-supplied step before slicing
step = int(input("Enter step: "))
if step == 0:
    print("Step cannot be zero")
else:
    print(s[:step])
```

Mistake 7 — Building queries by joining with user input directly

```
# Wrong: string concatenation into a query opens injection vulnerabilities
name = input("Enter name: ")
query = "SELECT * FROM students WHERE name = '" + name + "'"

# Correct: use parameterised queries
import sqlite3
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute("SELECT * FROM students WHERE name = ?", (name,))
```

Production Patterns

Strings in FastAPI Request Validation

FastAPI uses Pydantic v2 to validate string fields at the API boundary. Constraints like `min_length`, `max_length`, and `pattern` are declared with `Field()`.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class StudentRequest(BaseModel):
    name: str = Field(min_length=2, max_length=100, strip_whitespace=True)
    course: str = Field(min_length=2, max_length=60)
    roll_number: str = Field(pattern=r'^\d{4}[A-Z]{2}\d{3}$')

@app.post("/enroll")
async def enroll(req: StudentRequest):
    return {
        "message": f"Enrolled {req.name} in {req.course}",
        "roll": req.roll_number,
    }
```

Strings in Data Pipelines

```
# Clean and normalise a batch of raw records from a CSV
raw_records = [
    " Rohit Sharma | engineer | 5 ",
    "Tanvi Mehta|data scientist|3",
    " Karan Singh | analyst | 2 ",
]

def parse_record(line: str) -> dict:
    parts = [p.strip() for p in line.split("|")]
    return {
        "name": parts[0].title(),
        "role": parts[1].lower().replace(" ", "_"),
        "experience": int(parts[2]),
    }

records = [parse_record(r) for r in raw_records]
for rec in records:
    print(rec)
```

```
{'name': 'Rohit Sharma', 'role': 'engineer', 'experience': 5}
{'name': 'Tanvi Mehta', 'role': 'data_scientist', 'experience': 3}
{'name': 'Karan Singh', 'role': 'analyst', 'experience': 2}
```

Strings in a Prompt Template Engine

```
class PromptTemplate:
    def __init__(self, template: str):
        self.template = template

    def render(self, **kwargs) -> str:
        result = self.template
        for key, value in kwargs.items():
            result = result.replace(f"{{{key}}}", str(value))
        return result

qa_prompt = PromptTemplate(
    "Context:\n{context}\n\nQuestion: {question}\nAnswer:"
)
```

```
context = "Python was created by Guido van Rossum in 1991."
question = "Who created Python?"

print(qa_prompt.render(context=context, question=question))
```

Strings in a CLITool

```
import sys

def format_report(title: str, rows: list[tuple]) -> str:
    separator = "-" * 50
    lines = [separator, f" {title.upper()} ", separator]
    for label, value in rows:
        lines.append(f" {label:<25} {value:>15}")
    lines.append(separator)
    return "\n".join(lines)

report = format_report("Exam Results", [
    ("Arjun Verma", "92 / 100"),
    ("Drishya Nair", "87 / 100"),
    ("Om Prakash", "79 / 100"),
])
print(report)
```

EXAM RESULTS

Arjun Verma	92 / 100
Drishya Nair	87 / 100
Om Prakash	79 / 100

String Formatting Approach Comparison

APPROACH	SPEED	SAFETY	BEST USE CASE
f-string	Fastest	Safe for known data	General code, logging
<code>str.format()</code>	Medium	Safe	Reusable templates, config rendering
<code>%</code> -formatting	Medium	Safe	Legacy codebases only
<code>string.Template</code>	Slower	Safe for user templates	Config files, user-supplied patterns
t-string (3.14+)	Varies	Best for untrusted input	SQL builders, HTML renderers
Jinja2 (library)	Varies	Sandboxed	HTML templates, email bodies

Modern Python Patterns

Walrus Operator with Strings

The walrus operator (`:=`) assigns and uses a value in the same expression. It removes repeated calls when searching or checking a condition.

```
# Without walrus -- calls find() twice
line = "ERROR: database connection failed after 3 retries"
if line.find("ERROR") != -1:
    pos      = line.find("ERROR")
    message = line[pos:]

# With walrus -- calls find() once
if (pos := line.find("ERROR")) != -1:
    message = line[pos:]
    print(message)
# ERROR: database connection failed after 3 retries
```

```
# Read and process lines until a blank line is found
lines = ["name: Arjun", "score: 95", "grade: A", "", "name: Tanvi"]
i = 0
while (line := lines[i].strip()):
    key, value = line.split(": ")
    print(f" {key.upper()} -- {value}")
    i += 1
```


match-case for String Routing

Structural pattern matching (Python 3.10+) replaces long `if/elif` chains for routing and classification.

```
def classify_grade(grade: str) -> str:
    match grade.upper():
        case "A" | "A+":
            return "Distinction"
        case "B":
            return "First class"
        case "C":
            return "Second class"
        case "D":
            return "Pass"
        case _:
            return "Fail"

for g in ["A+", "B", "C", "F", "x"]:
    print(f"  {g} -- {classify_grade(g)}")
```

```
# Route HTTP methods to handlers
def handle_request(method: str, path: str) -> str:
    match method.upper():
        case "GET":
            return f"Fetching resource at {path}"
        case "POST":
            return f"Creating resource at {path}"
        case "DELETE":
            return f"Deleting resource at {path}"
        case _:
            return f"Method {method} not allowed"

print(handle_request("get",    "/students"))
print(handle_request("POST",   "/students"))
print(handle_request("PATCH", "/students/42"))
```

Union Types and Type Hints with Strings

Python 3.10+ allows `str | None` instead of `Optional[str]`, making annotations shorter and more readable.

```

from dataclasses import dataclass

@dataclass
class Student:
    name: str
    roll_number: str
    email: str | None = None
    department: str = "General"

def greet(student: Student) -> str:
    contact = f" ({student.email})" if student.email else ""
    return f"Hello, {student.name}{contact} -- Dept: {student.department}"

s1 = Student("Harsha", "2021CS042", "harsha@college.edu", "Computer Science")
s2 = Student("Om", "2021ME009")

print(greet(s1))
print(greet(s2))

```

f-string Debug Specifier

```

# f"{variable=}" prints both the name and the value -- very useful during debugging
name = "Durga"
length = len(name)
upper = name.upper()

print(f"{name=} {length=} {upper=}")
# name='Durga' length=5 upper='DURGA'

```

Older Style vs Modern Style

OLDER PATTERN	MODERN EQUIVALENT	WHY IT IS BETTER
<code>"Hello, " + name + "!"</code>	<code>f"Hello, {name}!"</code>	No manual concatenation
<code>"val=" + str(x)</code>	<code>f"val={x}"</code>	No explicit <code>str()</code> cast needed
<code>"%.2f" % price</code>	<code>f"{price:.2f}"</code>	Cleaner syntax, same result
<code>for i in range(len(s)):</code>	<code>for i, ch in enumerate(s):</code>	Avoids index errors, more Pythonic
<code>Optional[str]</code> in type hints	<code>str None</code>	Shorter, native syntax (3.10+)
<code>if s.find(x) != -1</code> then <code>s.find(x)</code>	<code>if (pos := s.find(x)) != -1</code>	Calls <code>find()</code> only once
<code>str.format()</code> with user data	<code>t"..."</code> template (3.14+)	Lets you sanitise before rendering

Quick Reference

OPERATION	SYNTAX / METHOD	NOTES
Create string	<code>s = "..."</code> / <code>s = '...'</code>	Triple quotes for multiline
Single char	<code>ch = 'a'</code>	Same type as <code>str</code> — no <code>char</code> type in Python
Length	<code>len(s)</code>	Includes spaces and punctuation
Index (positive)	<code>s[0]</code> , <code>s[2]</code>	Left to right; starts at 0
Index (negative)	<code>s[-1]</code> , <code>s[-3]</code>	Right to left; -1 is last character
Slice	<code>s[begin:end:step]</code>	End is exclusive; step=0 raises ValueError
Reverse	<code>s[::-1]</code>	Most idiomatic approach
Concatenate	<code>s1 + s2</code>	Both must be <code>str</code>
Repeat	<code>s * n</code>	One must be <code>int</code>
Membership	<code>sub in s</code>	Returns <code>bool</code>
Strip (both)	<code>s.strip()</code>	Removes leading and trailing whitespace
Strip (left)	<code>s.lstrip()</code>	Removes leading whitespace only
Strip (right)	<code>s.rstrip()</code>	Removes trailing whitespace only
Find	<code>s.find(sub)</code>	Returns -1 if not found
Find from right	<code>s.rfind(sub)</code>	Searches right to left; returns -1 if missing
Index	<code>s.index(sub)</code>	Raises <code>ValueError</code> if not found
Count	<code>s.count(sub)</code>	Non-overlapping occurrences
Replace	<code>s.replace(old, new)</code>	Returns new string; original is unchanged
Split	<code>s.split(sep)</code>	Default separator is any whitespace
Join	<code>sep.join(iterable)</code>	All items must be <code>str</code>
Upper / Lower	<code>s.upper()</code> / <code>s.lower()</code>	Returns new string
Swap case	<code>s.swapcase()</code>	Upper↔lower for each character
Title case	<code>s.title()</code>	First char of each word capitalised

OPERATION	SYNTAX / METHOD	NOTES
Starts with	<code>s.startswith(prefix)</code>	Accepts a tuple of prefixes
Ends with	<code>s.endswith(suffix)</code>	Accepts a tuple of suffixes
isalpha	<code>s.isalpha()</code>	True if all chars are letters
isdigit	<code>s.isdigit()</code>	True if all chars are digits
isalnum	<code>s.isalnum()</code>	True if all chars are letters or digits
islower	<code>s.islower()</code>	True if all cased chars are lowercase
isupper	<code>s.isupper()</code>	True if all cased chars are uppercase
isspace	<code>s.isspace()</code>	True if non-empty and all whitespace
istitle	<code>s.istitle()</code>	True if string is in title case
f-string	<code>f"value is {x}"</code>	Fastest formatting; preferred approach
Debug f-string	<code>f"{x=}"</code>	Prints variable name and value (3.8+)
format()	<code>"value is {}".format(x)</code>	Flexible; good for reusable templates
Thousands sep	<code>f"{n:,}"</code>	Inserts commas every 3 digits
Float precision	<code>f"{x:.2f}"</code>	Two decimal places
Zero pad	<code>f"{n:05d}"</code>	Pads with zeros to width 5
Alignment	<code>f"{s:<10}"</code> <code>f"{s:>10}"</code> <code>f"{s:^10}"</code>	Left, right, centre alignment
t-string (3.14+)	<code>t"value is {x}"</code>	Returns Template object; safe rendering
Immutability	All methods return a new string	Always reassign to keep the result
Safe concat	<code>"".join(list_of_strings)</code>	O(n); always prefer over + in a loop